ITA0610-Machine Learning

LAB EXPERIMENTS

Vasantha Nithi D Y

192424137

Lab 3: ID3 Decision Tree Algorithm from Scratch

Aim

To implement the **ID3 Decision Tree Algorithm** from scratch and classify the given training data based on the attribute with the highest information gain.

Algorithm

1. Create the training dataset.
2. Calculate the entropy of the target variable.
3. Calculate the information gain for each attribute.
4. Select the attribute with the highest information gain as the decision node.
5. Split the dataset based on the selected attribute.
6. Repeat the process recursively for the remaining attributes.
7. Stop when all examples belong to the same class or no attributes remain.
8. Display the generated decision tree.

Program

```
# Lab 3: ID3 Decision Tree Algorithm from Scratch


import pandas as pd

import math


# Create dataset

data = pd.DataFrame({

   "Outlook": ["Sunny", "Sunny", "Overcast", "Rain",

            "Rain", "Rain", "Overcast", "Sunny",

            "Sunny", "Rain"],


   "Temperature": ["Hot", "Hot", "Hot", "Mild",
```

```
            "Cool", "Cool", "Cool", "Mild",
            "Cool", "Mild"],

    "Humidity": ["High", "High", "High", "High",
            "Normal", "Normal", "Normal", "High",
            "Normal", "Normal"],

    "Wind": ["Weak", "Strong", "Weak", "Weak",
            "Weak", "Strong", "Strong", "Weak",
            "Weak", "Weak"],

    "Play": ["No", "No", "Yes", "Yes",
            "Yes", "No", "Yes", "No",
            "Yes", "Yes"]
})

# Calculate entropy
def entropy(column):
    values = column.value_counts()

    total = len(column)
    entropy_value = 0

    for count in values:
        probability = count / total
        entropy_value -= probability * math.log2(probability)

    return entropy_value



# Calculate information gain
```

```
def information_gain(data, attribute, target):
    total_entropy = entropy(data[target])

    values = data[attribute].unique()
    weighted_entropy = 0

    for value in values:
        subset = data[data[attribute] == value]
        probability = len(subset) / len(data)

        weighted_entropy += probability * entropy(subset[target])

    return total_entropy - weighted_entropy



# Build ID3 tree
def id3(data, attributes, target):

    # If all examples belong to one class
    if len(data[target].unique()) == 1:
        return data[target].iloc[0]

    # If no attributes remain
    if len(attributes) == 0:
        return data[target].mode()[0]

    # Find attribute with highest information gain
    gains = {
        attribute: information_gain(data, attribute, target)
        for attribute in attributes
    }
```

```
    best_attribute = max(gains, key=gains.get)

    tree = {best_attribute: {}}

    # Create branches
    for value in data[best_attribute].unique():

        subset = data[data[best_attribute] == value]

        remaining_attributes = [
            attribute
            for attribute in attributes
            if attribute != best_attribute
        ]

        tree[best_attribute][value] = id3(
            subset,
            remaining_attributes,
            target
        )

    return tree


attributes = ["Outlook", "Temperature", "Humidity", "Wind"]

tree = id3(data, attributes, "Play")

print("Decision Tree:")
print(tree)
```

Sample Output

Decision Tree:

```
{'Outlook': {
   'Sunny': {
      'Humidity': {
         'High': 'No',
         'Normal': 'Yes'
      }
   },
   'Overcast': 'Yes',
   'Rain': {
      'Wind': {
         'Weak': 'Yes',
         'Strong': 'No'
      }
   }
}}
```

Result

Thus, the **ID3 Decision Tree Algorithm was successfully implemented from scratch**, and a decision tree was generated by selecting attributes based on **Information Gain**.